

Cloud infrastructure

[Menu](#)[On this page](#)

Séance 9

Le temps estimé pour cette dernière séance est de... 3 séances.

Exercice de synthèse - Déploiement d'une application WordPress avec MySQL sur Kubernetes

INFO

Objectifs À la fin de cette séance, vous serez capable de :

- Identifier les composants d'une architecture WordPress sur Kubernetes
- Reconnaître les différents types d'objets Kubernetes (Deployment, StatefulSet, Service, Ingress)
- Expliquer la différence entre une application stateless et stateful
- Décrire le rôle des Secrets et ConfigMaps dans la gestion de configuration
- Créer une image Docker personnalisée basée sur WordPress
- Rédiger des manifestes YAML pour déployer une application multi-tiers
- Configurer des volumes persistants pour une base de données
- Distinguer les besoins en haute disponibilité pour WordPress et MySQL
- Examiner les dépendances entre les composants de l'application
- Vérifier le bon fonctionnement de l'application déployée
- Valider la conformité de l'architecture avec les spécifications techniques
- Concevoir une architecture complète d'application web sur Kubernetes
- Synthétiser l'ensemble des concepts vus pour déployer une solution fonctionnelle

Vous allez déployer une application WordPress complète avec sa base de données MySQL sur un cluster Kubernetes. Ce projet fait appel à l'ensemble des concepts vus dans les séances précédentes.

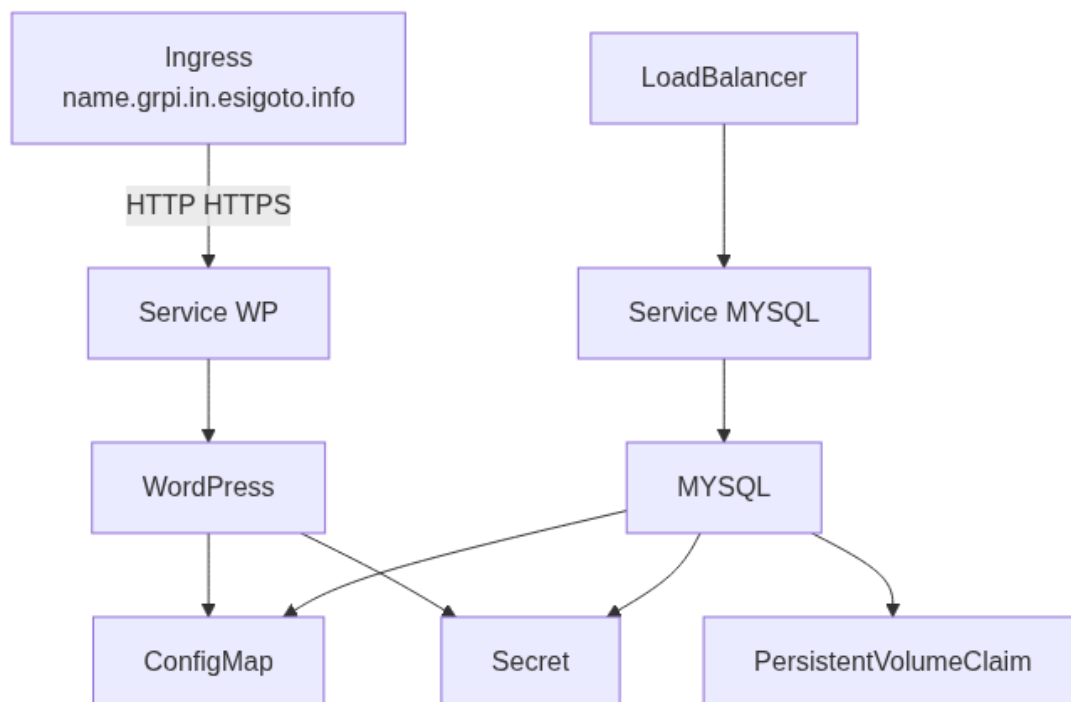
L'application sera composée de :

- l'application proprement dite, **WordPress** (*stateless*)

- une base de donnée associée, **MySQL** (*stateful*)
 - un système de stockage persistant pour MySQL
- une exposition externe (via *Ingress*)

Aspects techniques

L'architecture — avec une vue « haut niveau » — se présente comme suit :



Tâches

Déployer sur Kubernetes une application **WordPress** complète avec **MySQL**. Utiliser un *Deployment* (≥ 2 pods) et un *Service* pour *WordPress* (image personnalisée). Configurer un *StatefulSet* MySQL avec *PVC* et *Service* exposé en *LoadBalancer*. Mettre en place des *Secrets/ConfigMaps* injectés via variables d'environnement. Exposer WordPress via un *Ingress* sous `<shortname>.grp.5clo1r.in.esigoto.info` où `<shortname>` est un placeholder pour votre identifiant utilisateur.

Exigences
Utiliser un <i>Deployment</i> pour Wordpress (application stateless)
Associer un <i>Service</i> au <i>Deployment</i> (format du nom DNS : <code><svc>.<ns>.svc.cluster.local</code>)
Minimum 2 pods pour assurer la « haute disponibilité »
Utiliser un <i>StatefulSet</i> pour MYSQL (les données sont persistantes)
Associer un <i>Service</i> au <i>Statefulset</i> (format du nom DNS : <code><svc>.<ns>.svc.cluster.local</code>)
Prévoir un accès externe via <i>LoadBalancer</i> avec votre IP et un PORT disponible de votre range
Associer un volume persistant (<i>PersistentVolumeClaim</i>)
Stoker les mots de passe et <i>credentials</i> MYSQL dans un <i>Secret</i>
Stocker les variables d'environnement dans un <i>ConfigMap</i>
Transmettre ces valeurs aux pods <i>via</i> des variables d'environnement
Configurer un Ingress pour exposer le service WordPress
Utiliser un nom de domaine dans <code><votre-nom>.grp.5clo1r.in.esigoto.info</code>
Créer votre propre image Docker avec une couche supplémentaire contenant un plugin ou un thème personnalisé pour Wordpress

Note

La création de votre image nécessitera de la rendre disponible sur un *repository Docker*. Ceci peut se faire facilement par la création d'un compte sur [Docker Hub](https://hub.docker.com/).

Livrables

Vous fournirez :

- le schéma d'architecture (bas niveau)
- le Dockerfile de l'image WordPress personnalisée
- l'ensemble des manifestes YAML

Questions

Quand WordPress est déployé avec 2 replicas, l'interface d'administration peut poser des problèmes. Pourquoi ? Quelles sont – selon vous – les différentes pistes pour résoudre ces problèmes et permettre à l'interface d'administration de fonctionner sur un déploiement avec plusieurs replicas, tout en garantissant la cohérence et l'accès aux données et fichiers nécessaires ?

Remarque

Vous serez attentif et attentive à respecter une certaine **convention de nommage** pour tous vos objets Kubernetes.

Par exemple :

- `monwordpress-deployment`
- `monwordpress-mysql-statefulset`
- `monwordpress-db-secret`
- `monwordpress-config`

Documentation

Outre la documentation Kubernetes habituelle, la documentation des images Docker officielles sera bien utile. Par exemple :

- [Pour l'image Wordpress](#)
- [Pour l'image MySQL](#)

Mise à jour le: lundi 8 décembre 2025 à 11:17

Previous page
[Séance 8](#)

Next page
[Organisation](#)